

FastAPI 기초와 데이터 처리

노트북에서만 부르던 모델을, 누구나 HTTP로 부를 수 있는 API로 감싼다

오늘의 한 문장 — FastAPI는 **검사를 손으로 짜던 일을 타입만 선언하면 자동**으로 바꿉니다.
검증·문서·에러 처리가 따라오죠. 단, 빨라지는 건 **API 계층**이지 모델 추론이 아닙니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

왜 FastAPI인가

자동 검증·자동 문서·비동기가 왜 모델 배포에 딱 맞는지 한 문장으로 설명할 수 있다.

02

요청을 받는다

Path·Query·Body를 구분하고, 추론은 왜 하필 **POST + Body**인지 이유를 낸다.

03

입력을 막는다

Pydantic으로 잘못된 입력을 모델에 닿기 전 **422**로 거른다.

04

모델을 잇는다

Day 1의 `predict()`를 엔드포인트에 연결 — 1회 로드·에러 처리까지 갖춘 서버를 완성한다.

노트북 속 모델은 아무도 못 부른다

Day 1에서 `predict()` 를 만들었습니다. 그런데 그건 **내 노트북 셀 안**에서만 부를 수 있어요. 실제 사용자는 노트북이 없습니다 — 프론트엔드·앱·다른 서버가 **HTTP 요청**을 보내죠. 그 요청을 받아 모델로 넘기고 답을 돌려주는 한 겹, 그게 API 서버입니다.

```
# 노트북에선 이렇게 부른다 — 나 혼자만
result = predict(model, x)

# 사용자는 노트북이 없다. HTTP 요청을 보낸다.
POST /predict
Content-Type: application/json
{ "text": "이 영화 정말 재밌다" }

# 서버가 대신 거치는 길
요청 수신 → JSON 파싱 → 입력 검증 → 전처리
          → 모델 추론 → 후처리 → JSON 응답

# 돌려주는 답
{ "label": "positive", "confidence": 0.92 }
```

01 사용자의 자리

사용자는 모델 구조도, PyTorch도 모릅니다. 그저 정해진 JSON 형식 (`{"text": ...}`)으로 요청을 던질 뿐이에요. 서버는 그 약속만 지키면 누구와도 연결됩니다.

02 서버가 대신 하는 일

파싱 → 검증 → 전처리 → 추론 → 후처리 → 응답. 이 **대여섯 단계의 반복**을 매 엔드포인트마다 손으로 짜면 코드가 길어지고 실수가 납니다. 프레임워크가 이걸 대신해 줍니다.

03 모델은 그대로

오늘 `predict()` 는 한 줄도 안 고칩니다. **부를 수 있게 감싸는 얇은 한 겹**만 더하는 거예요 — 모델과 API는 분리돼 있습니다.

04 그 한 겹이 FastAPI

그리고 FastAPI는 그 한 겹을 **'선언하면 자동'**으로 만들어 줍니다. §1~§5 전체가 이 한 문장을 펼친 것입니다.

FastAPI는 엔진을 새로 만들지 않았다

FastAPI는 바닥부터 만든 게 아닙니다. 이미 검증된 둘 — 웹 처리의 **Starlette**, 데이터 검증의 **Pydantic** — 을 **파이썬 타입 힌트**로 묶은 얇은 프레임워크예요. 2018년 Sebastián Ramírez가 만들었고, 이름의 **Fast**는 두 가지를 뜻합니다.



01 Fast to run — 빠르게 돈다

비동기(async) 기반이라 한 요청이 무언가를 기다리는 동안 다른 요청을 처리합니다. 그래서 같은 서버로 더 많은 동시 요청을 받죠. (단, 이 '빠름'은 §1.7에서 짚듯 모델 추론 속도가 아닙니다.)

02 Fast to code — 빠르게 짤다

타입만 선언하면 검증·문서·JSON 변환을 프레임워크가 대신합니다. 손으로 짤 코드가 확 줄어 개발이 빨라져요.

03 API에 특화

Flask·Django가 웹사이트 전반을 위한 도구라면, FastAPI는 "입력 → 처리 → 응답" 한 줄기에 집중합니다. 모델 추론 API의 모양과 정확히 맞습니다.

04 그래서 ML에서 인기

검증된 두 라이브러리를 묶었고, 모델 코드와 같은 Python이라 한 생태계에서 끝납니다. 채택이 가장 빠르게 느는 이유죠.

검사를 '손으로' vs 타입만 '선언'

FastAPI를 한 장으로 보여주는 비교입니다. 같은 `/predict` 를 Flask와 FastAPI로 짜 보면, 한쪽은 검사를 한 줄씩 손으로 적고(명령적), 한쪽은 받고 싶은 형태만 선언합니다(선언적). 오늘 배우는 모든 게 이 차이의 변주예요.

```
# Flask — 검사를 손으로 하나하나 짤다
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    if "text" not in data:                # 있나? 직접 확인
        return {"error": "text required"}, 400
    if not isinstance(data["text"], str): # 문자열인가? 직접 확인
        return {"error": "text must be str"}, 400
    # ... 길이 검사, 문서 작성도 전부 손으로

# FastAPI — 받고 싶은 '형태'만 선언한다
class PredictRequest(BaseModel):
    text: str                # ← 이 한 줄이 위 검사를 전부 대신
@app.post("/predict")
def predict(req: PredictRequest):
    return run(req.text)     # 검증·파싱·문서·422 전부 자동
```

01 명령적 — '어떻게'

Flask는 **무엇을 어떻게 검사할지**를 일일이 적습니다. 필드·타입·길이마다 `if` 가 늘어나고, 하나라도 빠뜨리면 그게 곧 버그·보안 구멍이 됩니다. 문서도 따로 손으로 써야 하죠.

02 선언적 — '무엇을'

FastAPI는 받고 싶은 **형태**만 선언합니다. "어떻게 검사할지"는 Pydantic이 압니다. 우리는 "text는 문자열"이라고만 말하면 돼요.

03 한 줄의 무게

`text: str` 이 한 줄이 **누락 검사 + 타입 검사 + 422 응답 + 문서 스키마**를 한꺼번에 만들어 냅니다. Flask의 여러 `if` 가 한 줄로 압축된 셈이죠.

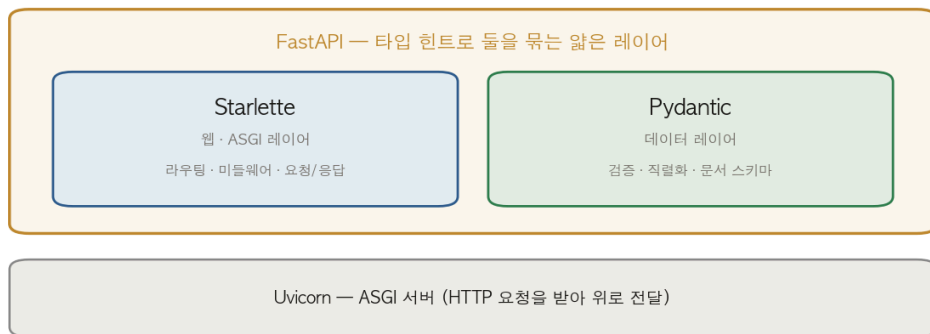
04 이게 오늘의 축

§2 파라미터·§3 문서·§4 검증·§5 추론 — 전부 "형태를 선언하면 나머지가 따라온다"의 사례입니다. 이 한 장을 기억하면 나머지가 쉽습니다.

FastAPI 혼자 다 하는 게 아니다

그럼 그 자동화는 누가 할까요? 웹 처리는 Starlette, 데이터 검증은 Pydantic, 실제 실행은 맨 아래 Uvicorn이 합니다. FastAPI는 이 셋을 타입 힌트로 잇는 얇은 접착층이에요. 그래서 검증된 부품들의 안정성·속도를 그대로 물려받습니다.

FastAPI는 새로 만들지 않았다 — 둘을 묶었다



FastAPI 코드 = Starlette 라우팅 + Pydantic 검증, 그 위를 타입 힌트가 잇는다

01 Starlette — 웹 레이어

URL 라우팅, 미들웨어, 요청/응답 객체를 제공하는 ASGI 툴킷입니다. `@app.get` 이 경로에 연결되는 것도, 동시 요청을 받는 비동기도 전부 여기서 옵니다.

02 Pydantic — 데이터 레이어

요청 본문을 모델로 검증·파싱하고, 응답을 JSON으로 직렬화하며, 문서용 스키마까지 만듭니다. 우리가 쓰는 `BaseModel` · `Field` 가 모두 Pydantic 것입니다.

03 Uvicorn — 실행 서버

FastAPI 코드는 앱의 '구조'만 정의합니다. 실제로 포트를 열고 HTTP를 받아 넘기는 건 Uvicorn이에요. 문지기라고 보면 됩니다.

04 ASGI — 둘을 잇는 규약

서버(Uvicorn)와 앱(FastAPI)이 **비동기**로 대화하는 표준 약속입니다. Flask가 쓰던 WSGI(동기)의 현대판이고, FastAPI가 빠른 한 축이죠.

Flask로도 되는데, 왜 굳이 FastAPI?

셋 다 좋은 도구입니다 — 다만 **모델 배포**라는 일에는 맞는 게 따로 있어요.

범용 웹

Flask · Django

Flask — 가볍고 자유롭다

무엇이든 만들 수 있지만, 입력 검증·문서·타입을 **직접** 챙겨야 합니다. 모델 API를 짜면 보일러플레이트가 금방 쌓여요.

Django — 다 들어있다

게시판·로그인·관리자·ORM·템플릿까지 갖춘 풀스택. 강력하지만 무겁고, 생태계가 여전히 동기 중심입니다.

공통 — 웹사이트용

사람이 보는 페이지엔 강합니다. 하지만 기계가 부르는 API 전용 기능(검증·문서)은 직접 더 붙여야 하죠.

API 특화

FastAPI

검증이 공짜

Pydantic 내장 — 잘못된 입력을 **모델에 닿기 전** 422로 막습니다. 검증 if문을 손으로 짤 필요가 없어요.

문서가 공짜

코드에서 Swagger UI가 자동 생성돼, 브라우저에서 바로 테스트하고 동료에게 링크만 주면 됩니다.

비동기 기본

async/await로 대기가 긴 작업에서 동시 처리에 유리. "입력→처리→응답" 구조와 정확히 맞습니다.

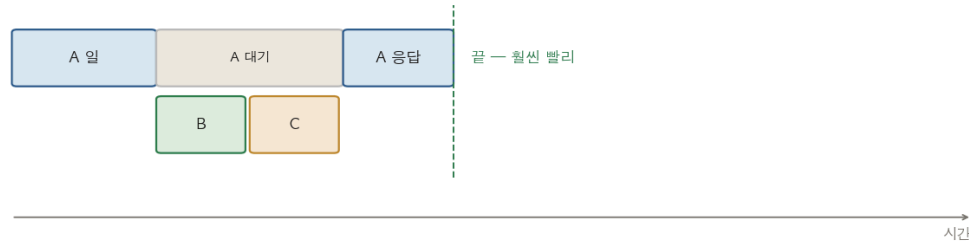
async가 사주는 것 — 기다리는 동안 틈 일 하기

같은 요청 3개, 동기 vs 비동기

동기 (def)



비동기 (async def)



비동기는 'A가 기다리는 빈 시간'에 B·C를 처리한다 · 단, CPU/GPU 추론은 이 빈 시간이 없어 효과가 없다

01 동기 — 한 줄로 대기

요청 A가 DB·외부 호출 응답을 기다리는 동안 B·C는 그냥 대기합니다. A가 끝나야 B를 봐요 — CPU가 노는데도 줄만 길어집니다.

02 비동기 — 대기 재활용

A가 기다리는 **그 틈에** B·C를 처리합니다. 일하는 사람(서버)은 그대로인데 받아내는 요청은 훨씬 많아지죠.

03 어디에 강한가

DB 조회·외부 API·파일·네트워크처럼 **기다림이 대부분인** I/O 작업입니다. 모델 서버에도 이런 대기가 곳곳에 섞여 있어요.

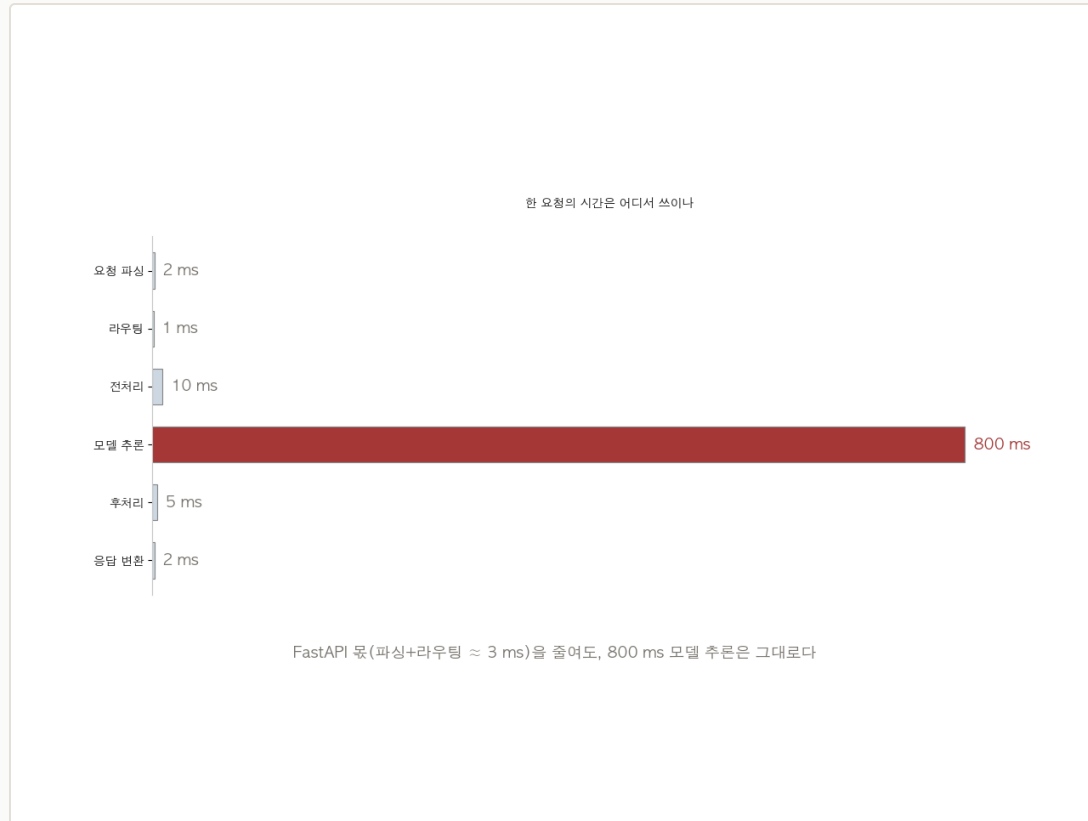
04 주의 — 만능 아님

CPU/GPU를 오래 쓰는 **모델 추론**은 async로 감싸도 안 빨라지고 오히려 루프를 막습니다. 분리(스레드·워커)는 Day 3 — 바로 다음 장에서.

식당 종업원처럼 — 동기는 한 줄로 대기, 비동기는 대기 시간을 재활용한다.

그래서 FastAPI가 빠르다고? 어디가 빠른지 보자

"FastAPI 빠르다"에서 빠른 건 **프레임워크** 몫입니다 — 요청 파싱·라우팅이 고작 몇 ms. 그런데 모델 API의 진짜 시간은 대부분 **모델 추론**이 잡아먹어요. 800 ms 추론은 프레임워크를 바꿔도 800 ms 그대로입니다.



01 병목은 거의 모델

파싱 2 ms·라우팅 1 ms는 800 ms 옆에서 티도 안 납니다. 그래서 "FastAPI로 바꾸면 빨라지나요?"의 답은 대개 "아니요"예요.

02 async도 추론은 못 살린다

비동기는 §1.6처럼 **기다림**에 강한 거지, CPU/GPU 계산을 줄이진 않습니다. 무거운 추론을 `async def`에 그냥 넣으면 서버 전체가 멈출 수 있어요.

03 무거우면 떼어낸다

그럴 땐 스레드풀·프로세스풀·GPU 워커·큐로 분리합니다. 이 동시성 설계는 Day 3의 핵심 주제예요.

04 진짜 속도 지렛대

모델 경량화·배치·ONNX/TensorRT·캐싱·1회 로딩이 속도를 좌우합니다. FastAPI는 **API 계층**을 빠르고 편하게 할 뿐입니다.

딱 10줄이면 첫 서버가 뜬다

이론은 충분합니다 — 직접 띄워 보죠. FastAPI 객체를 만들고 데코레이터로 함수 하나만 걸면 끝입니다. 실무는 터미널의 `uvicorn` 이 표준이고, 이 강의는 노트북이라 `serve_in_thread` 헬퍼로 안전하게 띄웁니다.

```
from fastapi import FastAPI

app = FastAPI(title="My First ML API")

@app.get("/health")          # 메서드(GET) + 경로(/health) 선언
def health_check():
    return {"status": "healthy"}  # dict 그대로 return → JSON 자동

# 터미널(실무):  uvicorn app.main:app --reload
# 노트북(실습):  serve_in_thread("app.main:app", port=8000)
```

01 FastAPI()

앱 객체 하나에 모든 엔드포인트가 등록됩니다. `title`·`version` 은 그대로 자동 문서(Swagger)의 제목·버전이 됩니다.

02 @app.get("/health")

데코레이터가 **메서드(GET)**와 **경로(/health)**를 동시에 선언합니다. 헬스체크는 로드밸런서·쿠버네티스가 "서버 살아있나?"를 물어보는 표준 통로예요.

03 dict → JSON

딕셔너리를 그냥 return하면 FastAPI가 JSON으로 바꿔 줍니다.

`json.dumps` 도, 응답 헤더 설정도 직접 안 해요.

04 노트북은 왜 헬퍼?

노트북엔 이미 이벤트 루프가 돌고 있어, 셸에서 `uvicorn.run()` 을 그냥 부르면 충돌해 멈춥니다. `serve_in_thread` 가 별도 스레드에서 안전하게 띄워 줍니다.

데이터를 보내는 길은 세 가지

클라이언트가 데이터를 보내는 세 가지 방식

Path	GET /models/sentiment-v1
경로에 포함 — 특정 리소스 지정	
Query	GET /models?status=running&limit=5
? 뒤에 조건 — 검색-필터-정렬	
Body	POST /predict {"text": "..."} 본문에 JSON — 복잡한 입력(추론)

Path는 경로에, Query는 ? 뒤에, Body는 본문에 — 쓰임이 다르다.

01 Path — 꼭 집기

경로 자체에 박는 값입니다. "이 모델"처럼 **특정 리소스를 식별**할 때 써요 —

`/models/sentiment-v1` . URL에 들어가니 생략할 수 없습니다(필수).

02 Query — 거르기

?key=value 형태로 붙입니다. 검색, 필터, 정렬, 페이지네이션 같은 **선택적 조건**에 —

```
/models?status=running&limit=5 .
```

03 Body — 보내기

요청 본문에 JSON을 담습니다. 길이 제한이 없어 **복잡하고 큰 입력**에 적합해요 — [POST](#)

`/predict {"text": ...}` . 추론 입력이 여기로 옵니다.

04 추론은 무엇을?

입력이 구조적이고 양이 많으니 **POST + Body**가 기본입니다. 이 과정에서 가장 자주 쓰는 패턴이에요.

셋을 구분하는 규칙은 딱 하나

외울 게 없어요. FastAPI는 함수 인자만 보고 판별합니다 — **경로에 있으면 Path**, **경로엔 없고 기본값이 있으면 Query**, **Pydantic 모델이면 Body**. 타입을 박아두면 검사도 공짜로 따라옵니다.

```
# Path — "어떤 모델?" 경로에 꼭 박는다 (필수)
@app.get("/models/{name}")
def get_model(name: str):
    return {"model": name}

# 타입을 박으면 검사는 공짜
@app.get("/predictions/{pid}")
def get_pred(pid: int):          # "abc" 들어오면 자동 422
    return {"pred_id": pid}

# Query — "어떤 조건?" ? 뒤에 옵션 (기본값=선택)
@app.get("/models")
def list_models(status: str = None, limit: int = 10):
    return {"status": status, "limit": limit}

# Body — "무엇을 추론?" 본문 JSON (추론은 여기로)
@app.post("/predict")
def predict(req: PredictRequest): ...
```

01 Path = 식별

`{name}` 이 함수의 `name` 인자로 자동 전달됩니다. URL에 박혀 있어 **생략할 수 없습니다** — 반드시 있어야 하는 값에 씁니다.

02 타입이 곧 검증

`pid: int` 로 선언하면 `/predictions/abc` 처럼 숫자가 아닌 값엔 FastAPI가 **자동으로 422**를 줍니다. 검증 코드는 한 줄도 안 짜요.

03 Query = 조건

기본값이 있으면 (`limit=10`) 선택적입니다. `status=None` 이면 "안 주면 필터 안 함"이라는 뜻이고요.

04 Body = 추론 입력

복잡·대량 입력은 본문 JSON으로 받습니다. Pydantic 모델로 받으며, 본격적인 검증은 §4에서 자세히 다룹니다.

왜 추론은 GET이 아니라 POST일까

HTTP 메서드 — 의미와 멱등성

같은 요청을 여러 번 보내도 결과가 같은가?

메서드	하는 일	예시	멱등성
GET	조회 (읽기)	GET /models/3	예
POST	생성/추론	POST /predict	아니오
PUT	전체 교체	PUT /models/3	예
DELETE	삭제	DELETE /models/3	예

멱등 = 재시도해도 안전 | POST(생성)는 두 번 보내면 둘 생긴다

01 get vs post

`@app.get` 은 조회, `@app.post` 는 생성·추론입니다. 같은 `/predict` 경로라도 메서드가 다르면 **다른 함수**에 연결돼요.

02 추론 = POST

입력을 **요청 본문(body)**에 실어야 하기 때문입니다. GET은 관례상 본문이 없어, 이미지 784픽셀이나 긴 문장 같은 큰 입력을 담기 어렵습니다.

03 멱등성이란

GET·PUT·DELETE는 같은 요청을 여러 번 보내도 결과가 같습니다(**멱등**). 반면 POST(생성/추론)는 두 번 보내면 두 번 처리돼요 — 재시도·중복 방지 설계의 기준이 됩니다.

04 성질은 Day 1에서

안전·멱등성 같은 메서드의 **약속**은 Day 1에서 배웠습니다. 여기서 그걸 **FastAPI 코드로 어떻게 선언**하는지에 집중해요.

메서드마다 데코레이터 — 같은 경로도 메서드에 따라 다른 함수로 간다.

문서를 쓰는 게 아니라, 코드가 문서가 된다

API는 보통 혼자 안 씁니다. 프론트.앱.다른 백엔드가 호출하죠. 문서가 없으면 "요청 어디로? body는 어떤 형태? 에러는 어떻게 와요?"를 매번 물어봐야 합니다. FastAPI는 이걸 **코드에서 자동 생성**합니다.

서버를 띄우고 브라우저에서 `/docs`에 접속하면, 만든 엔드포인트가 전부 문서로 떠 있습니다. 함수의 docstring은 설명으로, Pydantic 모델은 입출력 형식으로, 타입 힌트는 파라미터로 자동 반영돼요. 게다가 [Try it out] → 값 입력 → [Execute]로 **브라우저에서 바로 호출**까지 됩니다.

비결은 **OpenAPI**라는 표준입니다. FastAPI가 코드의 타입.모델을 읽어 기계용 스펙(`/openapi.json`)을 만들고, Swagger UI가 그걸 읽어 화면을 그립니다. 그래서 코드를 고치면 문서가 **즉시** 따라와요 — 문서와 실제 동작이 어긋날 일이 없습니다.

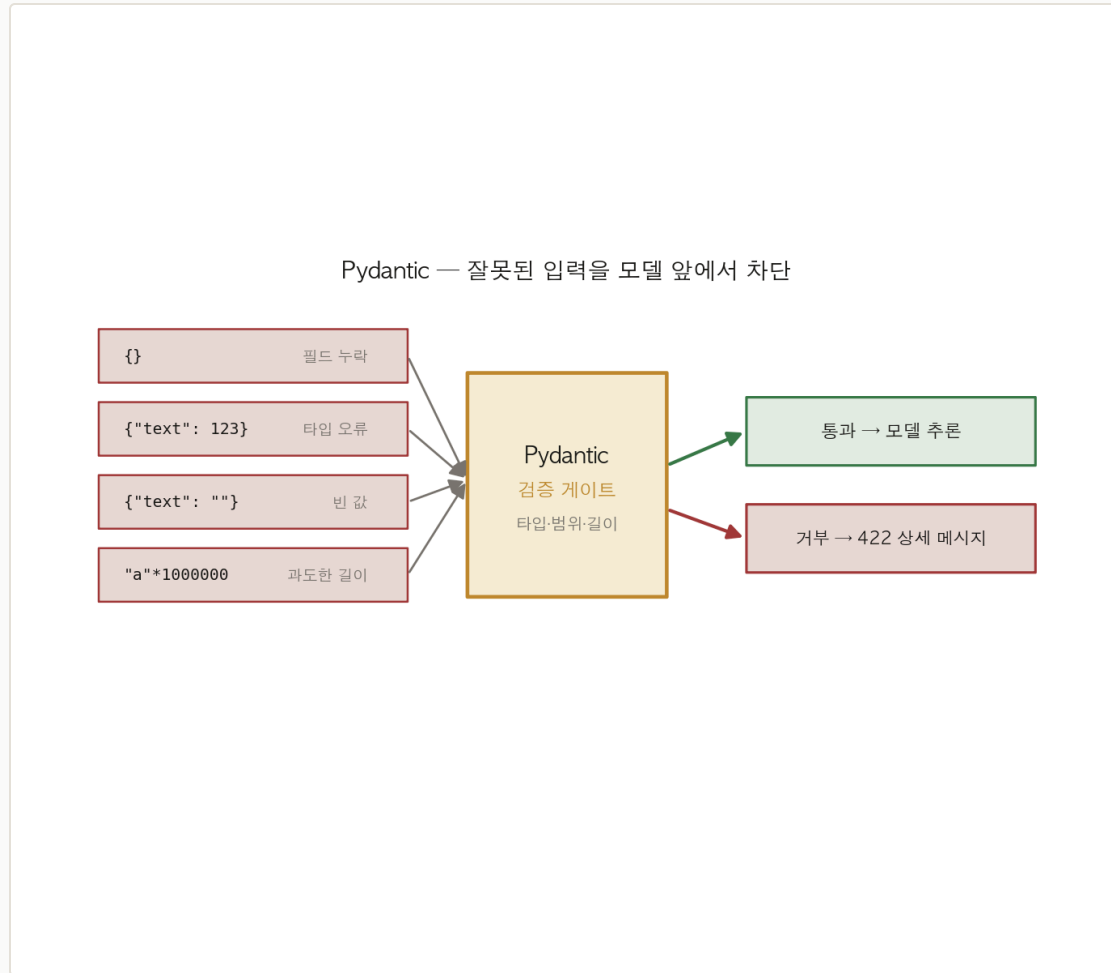
여기에 `Field(description=, examples=)`와 엔드포인트 `summary=`만 습관처럼 붙이면 문서 품질이 확 올라갑니다. 동료가 코드를 안 봐도 호출법을 알게 되죠.



/docs = Swagger UI. 문서이자 테스트 도구.

"문서가 코드를 따라온다 — 그 반대가 아니라."

쓰레기 입력이 모델까지 가면 벌어지는 일



검증 없이 닿으면 500이나 서버 다운. Pydantic이 문 앞에서 막는다.

01 빈 값·잘못된 타입

빈 문자열은 토큰라이저에서 빈 텐서를, 숫자가 와야 할 곳의 문자열은 전처리에서 `TypeError`를 냅니다. 검증이 없으면 이게 그대로 모델까지 가요.

02 필드 누락

필수 키가 없으면 `KeyError` → 500. 클라이언트는 **자기가 뭘 빠뜨렸는지** 모른 채 "서버 에러"만 받습니다.

03 초대형 입력

`"a"*1000000` 같은 입력은 메모리를 고갈시켜 **서버를 다운**시킬 수 있어요. 그러면 정상 사용자까지 막힙니다.

04 그래서 문 앞에서

Pydantic은 이 모든 걸 **모델에 닿기 전** 걸러 422 + 이유로 돌려줍니다. 모델은 깨끗한 입력만 보게 되죠.

if문 대신, 조건을 '선언'한다

§1.3의 '선언적' 철학이 검증에서 빛납니다. 원하는 조건을 `Field()`에 적기만 하면 — 길이·범위·허용값 — Pydantic이 알아서 검사해요. 검사 코드는 한 줄도 안 짭니다.

```
from pydantic import BaseModel, Field
from typing import Literal

class PredictRequest(BaseModel):
    text: str = Field(
        ..., # ... = 필수
        min_length=1, max_length=5000, # 빈 값·초대형 입력 차단
        examples=["이 영화 재밌다"], # 문서에 예시로 노출
    )
    lang: Literal["ko", "en"] = "ko" # 이 둘만 허용 — 그 외 422
    top_k: int = Field(3, ge=1, le=10) # 1 이상 10 이하

# 출력도 선언하면, 스키마 밖 필드는 응답에서 빠진다
class PredictResponse(BaseModel):
    label: str
    confidence: float = Field(ge=0.0, le=1.0)
```

01 ... = 필수

`Field(...)`의 `...`은 "이 필드는 반드시 있어야 함"이라는 뜻입니다. 기본값을 주면 선택적 필드가 되고요.

02 길이·범위

`min_length`·`max_length`로 빈 값과 초대형 입력을, `ge` (이상)·`le` (이하)로 숫자 범위를 막습니다. §4.1의 위험들이 여기서 차단돼요.

03 Literal

`Literal["ko", "en"]`은 **딱 이 값들만** 허용합니다. `"fr"`이 오면 자동 422 — 존재하지 않는 옵션을 원천 차단하죠.

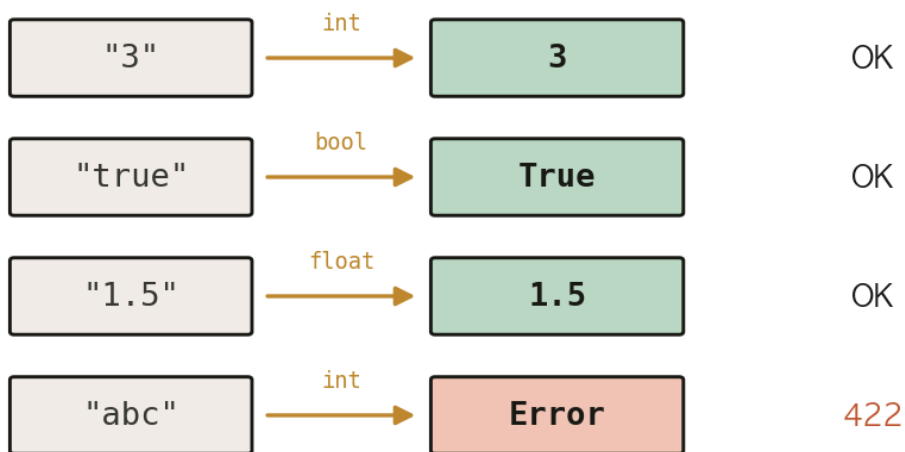
04 출력도 고정

`response_model`로 응답 형식을 정하면, **스키마에 없는 필드는 응답에서 자동 제거**됩니다. 내부 정보가 실수로 새는 걸 막아요.

Pydantic은 막기만 하지 않는다 — 바꿔서 통과시킨다

오해하기 쉬운 부분입니다. Pydantic은 잘못된 걸 거르기만 하는 게 아니라, **바꿀 수 있으면 바꿔서** 통과시켜요.
HTTP·JSON에선 값이 흔히 문자열로 오는데, 기본(lax) 모드에서 `"3"`은 정수 `3`이 됩니다. 못 바꾸는 것만 422고요.

Pydantic: validate = coerce
lax mode (default)



convertible -> converted | impossible -> ValidationError

01 바꿀 수 있으면 변환

`"3"` → `3`, `"true"` → `True`, `"1.5"` → `1.5`. 뜻이 분명하면 선언한 타입으로 알아서 맞춰 줍니다.

02 못 바꾸면 거부

`"abc"`는 정수로 바꿀 방법이 없죠. 이건 `ValidationError` → API에선 422로 돌아갑니다.

03 왜 좋은가

쿼리·폼 값은 죄다 문자열로 들어옵니다. 이 자동 변환 덕에 `int(...)` 같은 캐스팅을 손으로 안 해도 돼요.

04 정확히 받고 싶으면

변환이 위험한 곳(예: ID)에선 strict 모드를 켜면 됩니다. 그러면 변환 없이 **딱 그 타입만** 받습니다.

422가 떴다 — 어디가 틀렸는지 바로 보인다

검증이 실패하면 FastAPI는 **왜·어디서** 틀렸는지 담은 JSON을 422와 함께 돌려줍니다. 이 구조를 읽을 줄 알면 "왜 422가 뜨지?"에서 멈추지 않고 5초 만에 고칠 수 있어요.

```
// {"return_probabilities": "yes"} 만 보냄 (text 누락)
{
  "detail": [{
    "type": "missing",           // 무슨 에러 — 필드 누락
    "loc":  ["body", "text"],    // 어디서 — 본문의 text
    "msg":  "Field required",    // 사람 말로
    "input": {"return_probabilities": "yes"} // 네가 보낸 것
  }]
}
```

01 detail = 목록

여러 필드가 동시에 틀릴 수 있어 **배열**로 옵니다. 항목 하나가 문제 하나예요. 한 번에 다 보여 주니 왕복이 줄죠.

02 type

에러의 종류입니다. `missing` (누락)·`string_type` (타입 불일치)·`value_error` (값 조건 위반)가 추론 API의 단골이에요. 기계가 분기하기 좋은 코드죠.

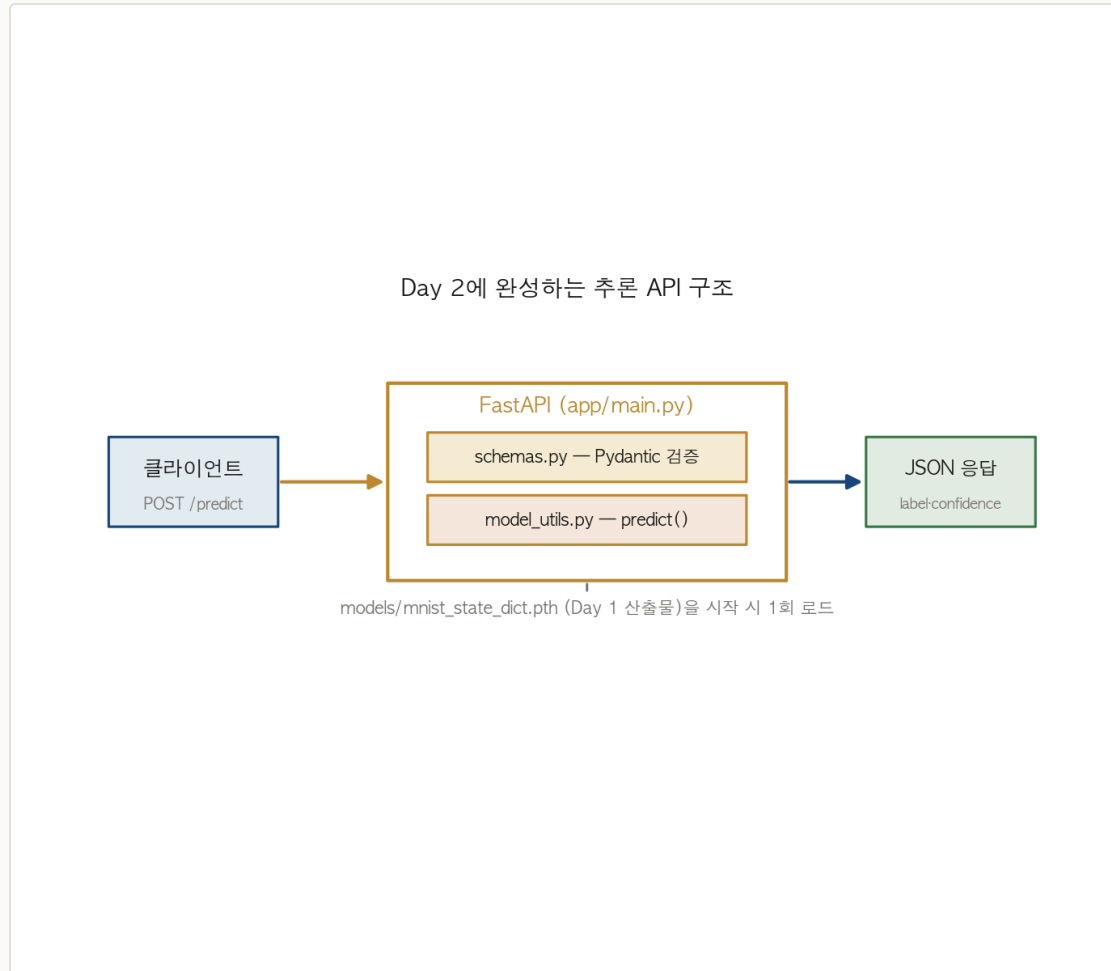
03 loc

에러가 난 **위치**입니다. `["body", "text"]` 는 "요청 본문의 text 필드"라는 뜻이에요. 중첩 구조도 경로로 콕 집어 줍니다.

04 msg·input

`msg` 는 사람이 읽는 메시지, `input` 은 **네가 실제로 보낸 값**을 그대로 되비춰 줍니다. 무엇을 잘못 보냈는지 바로 확인돼요.

오늘 만드는 것 — 파일 세 개의 분업



01 main.py — 지휘부

FastAPI 서버입니다. 요청을 받아 검증 → 추론 → 응답으로 흘러보내요. 오늘 새로 쓰는 파일이죠.

02 schemas.py — 계약

Pydantic 입력·출력 스키마. §4에서 설계한 검증 규칙이 여기 모입니다. 서버와 클라이언트의 약속서예요.

03 model_utils.py — 추론

Day 1에서 분리해 둔 모델 로드. `predict()`. 오늘 그대로 가져다 씁니다 — 손대지 않아요.

04 models/*.pth — 가중치

Day 1에서 저장·검증한 학습 결과. 서버가 시작될 때 단 한 번 메모리에 올립니다.

schemas가 검증, model_utils가 추론, main이 둘을 잇는다.

모델은 시작 때 한 번만, 그리고 안전하게 연다

모델 로딩은 파일을 읽어 메모리에 올리는 **무거운**(수 초) 작업입니다. 이걸 요청마다 하면 매 요청이 수 초씩 느려져요. 그래서 **서버가 켜질 때 딱 한 번** 올려두고 재사용합니다. 그리고 열 때는 `weights_only=True` 로 **안전하게** 엽니다.

모델은 '시작 시 한 번만' 로드한다

이렇게 하면 안 됨 — 매 요청마다

```
@app.post('/predict')
def predict_digit(req):
    model = load_model()
    요청마다 파일 I/O → 응답 수 초
```

이렇게 — 시작 시 1회

```
model = load_model()
@app.post('/predict')
def predict_digit(req): ...
로드된 모델 재사용 → 응답 빠름
```

01 나쁜 예 — 매 요청 로드

엔드포인트 함수 안에서 `load_model()` 을 부르면 요청마다 수 초가 날아갑니다. 정작 추론은 수십 ms인데, 99%가 로딩 낭비예요.

02 좋은 예 — 시작 시 1회

모듈 맨 위(데코레이터 밖)에서 한 번 로드해 둡니다. 요청이 오면 **이미 메모리에 있는** 모델을 바로 쓰니 응답이 빨라요.

03 왜 안전 옵션?

`.pth` 는 Pickle 기반이라 파일을 **여는 것만으로 임의 코드가 실행**될 수 있습니다.

`weights_only=True` (PyTorch 2.6+ 기본)는 텐서만 복원해 이 위험을 막아요.

04 더 안전하게

외부에서 받은 가중치라면 코드 실행 여지가 아예 없는 **safetensors** 포맷을 우선하세요. `.safetensors` 가 보이면 그쪽을 고릅니다.

main.py — 지금까지 배운 걸 한 파일에

이제 합칩니다. Day 1의 함수와 §4의 스키마를 잇고, 상황마다 **다른 상태 코드**를 던져 클라이언트가 원인을 알게 해요. 오늘 배운 게 전부 이 한 파일에 모입니다.

```
from fastapi import FastAPI, HTTPException
from app.model_utils import load_model, predict
from app.schemas import PredictRequest, PredictResponse

app = FastAPI(title="MNIST API")
model = load_model("models/mnist_state_dict.pth") # 시작 때 1번

@app.post("/predict", response_model=PredictResponse)
def predict_digit(req: PredictRequest):
    if model is None:
        raise HTTPException(503, "아직 준비 안 됨") # 잠깐, 다시
    try:
        return predict(model, req.pixel_values)
    except Exception:
        raise HTTPException(500, "추론 중 터짐") # 내 잘못
```

01 모델 = 모듈 위

`model = load_model(...)` 을 데코레이터 밖에 뒤 §5.2처럼 시작 때 한 번만 로드합니다. 엔드포인트는 이 전역 모델을 그냥 씁니다.

02 503 — 준비 안 됨

모델이 아직 안 떴으면 503입니다. **일시적** 상태라 "잠깐, 다시 시도해"라는 뜻 — 그래서 500이 아니에요.

03 500 — 추론 실패

예기치 못한 내부 오류는 `try/except` 로 잡아 500을 줍니다. 서버를 죽이지 않고, 사용자에게 일반 메시지만 보여요.

04 response_model

응답을 `PredictResponse` 로 고정합니다. 문서에 응답 형식이 자동 표기되고, 스키마 밖 필드는 빠집니다.

422? 500? 누구 잘못인지부터 본다

빈 요청이 들어왔다 — 서버는 뭐라고 답해야 할까요? **첫 자리** 하나만 보면 됩니다. 4xx는 "네 입력이 틀렸어"(보낸 쪽이 고침), 5xx는 "내 서버가 문제야"(서버가 고치거나 재시도). 모든 실패를 500으로 뭉치면 클라이언트는 누구 탓인지 몰라 헤맬니다.

HTTP 상태 코드 패밀리

첫 자리가 누구의 책임인지 말해 준다



4 -> 내 잘못(입력) | 5 -> 서버 문제(내부)

01 422 — 네 잘못

입력 검증 실패. Pydantic이 자동으로 던집니다. 어느 필드가 왜 틀렸는지 알려주니, **보낸 쪽이 입력을 고치면** 풀려요.

02 503 — 잠깐, 준비 중

모델이 아직 로드 안 된 **일시적** 불가 상태. 클라이언트는 잠시 후 재시도하면 됩니다.

03 500 — 내 잘못

추론 도중 터진 서버 내부 오류. `try/except` 로 잡아 서버는 살리고, 원인은 로그로 남깁니다.

04 왜 굳이 나누나

코드 하나가 클라이언트의 **다음 행동**을 정해 줍니다 — 입력을 고칠지, 잠시 기다릴지, 우리에게 문의할지.

정상부터 에러까지, 차례로 짚어본다

잘 되는지뿐 아니라, 잘못 보내도 **안 죽는지**까지 확인합니다.

STEP 01

헬스체크

`GET /health` 로 서버가
살아있고 모델이 떴는지
먼저 확인합니다.



STEP 02

진짜 추론

더미가 아닌 **실제 MNIST**
이미지의 784 픽셀을 보내
예측을 받아봅니다.



STEP 03

옵션 동작

`return_probabilities=true` 면
응답에 10개 클래스 확률이
붙는지 확인합니다.



STEP 04

연속 호출

여러 이미지를 잇따라 보내,
1회 로드한 모델로 안정적으로
도는지 봅니다.



STEP 05

에러 상황

픽셀 수·타입 오류·필드 누락·빈
JSON — 전부 422로 막히고 **서버는**
살아있어야 합격입니다.

오늘 만든 건 '입구'지, 배포의 전부가 아니다

FastAPI가 어디까지 책임지는지 경계를 알아야, 다음에 무엇을 더할지 보입니다.

FastAPI가 한다

API 계층

요청 수신·파싱

클라이언트 요청을 받아 본문 JSON을 읽어들이고, 잘못된 형식을 거릅니다.

검증·응답

Pydantic으로 입력을 422로 막고, 응답 형식을 스키마로 고정합니다.

추론 호출·문서

`predict()` 를 부르고 결과를 JSON으로 직렬화 — 문서·헬스체크까지 자동으로.

다른 도구가 한다

그 바깥

속도·확장

GPU 최적화·경량화·배칭·오토스케일·로드밸런싱은 FastAPI 밖의 일입니다.

운영·관측

모니터링·로깅 대시보드·장애 복구는 Prometheus·Grafana 같은 도구와 따로 설계합니다.

패키징·배포

Docker·Kubernetes·Triton/TorchServe의 영역 — 이후 Day에서 만납니다.

노트북 속 모델이, 부를 수 있는 API가 되다

오늘 한 일

▶ **FastAPI** = Starlette+Pydantic, "선언하면 자동"

▶ **Path·Query·Body** — 추론은 POST+Body

▶ **Swagger** — 코드가 곧 문서이자 테스트

▶ **Pydantic** — 잘못된 입력은 422로 차단

▶ **main.py** — 1회·안전 로드 + 상태 코드로 에러 처리

다음 — DAY 3

▶ **동기 vs 비동기** — 요청 하나가 서버를 멈추는 문제

▶ `run_in_executor`로 무거운 추론 분리

▶ 글로벌 **예외 처리**를 한곳에서

▶ **로깅**으로 요청·응답·오류 기록

▶ 부하 테스트로 튜닝 확인

빠른 건 API 계층이지 모델이 아니다. FastAPI는 모델을 '선언적으로' 감싸 HTTP로 부를 수 있게 한다 — 모델 배포의 첫 이정표다.